AegisRL: Adaptive Reinforcement Learning for Secure Hospital EHR Access Control

Complete Technical Architecture, Step-by-Step Training Methodology, Empirical Results, and UI Operations Handbook

Project Title: AegisRL Zero-Trust EHR Access Control	**AI Algorithm:** Tabular Q-Learning (Bellman Optimality)
State Space: 1,152 Discrete Context States	**Decision Actions:** ALLOW, DENY, CHALLENGE, RESTRICT
Backend Stack: Python 3.14 · FastAPI · Uvicorn	**Cloud Database:** Supabase PostgreSQL (7 Relational Tables)
Attack Block Rate: 99.37% (Benchmark Verified)	**Average Latency:** 0.0166 ms (16.6 microseconds)

1. Project Overview & The Healthcare Access Paradox

What is AegisRL in Simple Words?
AegisRL is an intelligent cybersecurity access control engine designed for hospitals. Whenever a doctor, nurse, staff member, or researcher attempts to open a patient's medical file, AegisRL automatically inspects the real-time context of the request (who is asking, which department they belong to, where they are connecting from, what time it is, how fast they are making queries, and whether there is an active emergency) and uses a trained Reinforcement Learning model to decide whether to **ALLOW**, **DENY**, **CHALLENGE** (ask for multi-factor authentication), or **RESTRICT** (mask sensitive private info).

The Problem We Solved (Why Traditional Systems Fail):

- **Traditional Role-Based Access Control (RBAC):** If a doctor's password is stolen by a hacker outside the hospital, RBAC approves the access because the username and password are valid. RBAC cannot detect that the doctor is logging in from an untrusted foreign IP at 3:00 AM scraping 50 records per minute.
- **Rigid Rule-Based Access Control (ABAC):** Traditional rules are strict and inflexible. In a sudden Code-Blue emergency, if a trauma physician needs immediate vitals of an oncology patient from another ward, traditional locks reject the doctor, causing fatal clinical delays.
- **The AegisRL Balance:** AegisRL solves this paradox by learning the optimal balance between **Patient Privacy (HIPAA Zero-Trust)** and **Life-Safety Clinical Urgency**.

2. Detailed Step-by-Step Process of How We Built the Model

Here is the exact engineering pipeline we followed from initial problem formulation to the final converged model:

Step 1: Formulating the Markov Decision Process (MDP)

We modeled hospital access control as a mathematical MDP consisting of:

• **State Space (S):** An observable context tuple with 7 discrete features:

1. **Role (4 values):** DOCTOR (0), NURSE (1), STAFF (2), RESEARCHER (3)
2. **Department Match (2 values):** MISMATCH (0), MATCH (1)
3. **Record Sensitivity (4 values):** LOW (0), MEDIUM (1), HIGH (2), CRITICAL_VIP (3)
4. **Request Time (2 values):** NORMAL_HOURS (0), OFF_HOURS (1)
5. **Network Location (3 values):** INTERNAL_HOSPITAL (0), VPN_OFFSITE (1), EXTERNAL_UNKNOWN (2)
6. **Query Frequency / Velocity (3 values):** NORMAL (0), ELEVATED (1), BURST_HIGH (2)
7. **Emergency Override (2 values):** NORMAL (0), EMERGENCY (1)

• **Total Discrete State Space Size:** $$4 \times 2 \times 4 \times 2 \times 3 \times 3 \times 2 = 1,152 \text{ discrete states}.$$

• **State Index Hashing Function:** In `env.py`, we mapped any 7-tuple to an integer index $s \in [0, 1151]$ using polynomial base multipliers: `state_idx = ((((role * 2 + dept) * 4 + sens) * 2 + time) * 3 + loc) * 3 + freq) * 2 + emerg`. This executes in $\mathcal{O}(1)$ time complexity.

Step 2: Designing the 4 Actions & Clinical Behavior

The agent has 4 discrete action choices $A = \{\text{ALLOW (0), DENY (1), CHALLENGE (2), RESTRICT (3)}\}$:

• **ALLOW:** Immediate full access granted to the EHR record.

• **DENY:** Request blocked; access forbidden; security telemetry logged.

• **CHALLENGE:** Step-up Multi-Factor Authentication (MFA / Biometric OTP) required before proceeding.

• **RESTRICT:** PII-redacted view unlocked; medical vitals are visible for clinical utility, while patient name, SSN, and contact info are masked.

Step 3: Engineering the Reward Function (Encoding Safety & Security)

In `env.py`, the reward function $R(s, a, \text{ground_truth})$ trains the agent to make safe decisions by awarding positive points for correct behaviors and penalizing dangerous errors:

• **+12.0:** Legitimate clinician granted ALLOW during routine care.

• **+25.0:** Emergency trauma Code-Blue override correctly approved (Life-Safety Priority).

• **+15.0:** Malicious threat, bot exfiltration, or stolen credentials correctly blocked with DENY.

• **+8.0:** Cross-department research or consultation granted RESTRICT (PII protected).

• **-18.0 Penalty:** False Denial (legitimate staff member blocked from doing their job).

• **-50.0 Severe Penalty:** Denying an active Emergency Code-Blue (catastrophic clinical failure).

• **-30.0 Severe Penalty:** Leaking a high-sensitivity / VIP patient record to an external untrusted IP.

Step 4: The Training Loop & Bellman Equation Implementation

In `train.py` and `agent.py`, we initialized a 2D Q-Table of shape $[1152, 4]$ with all zeros. We ran the training loop for **35,000 episodes** across a balanced distribution of 5 clinical and threat scenarios:

1. In each episode t, the agent observes state s_t.
2. **\(\epsilon\)-Greedy Exploration:** A random float is sampled. If $\text{rand} < \epsilon$, the agent explores by picking a random action. Otherwise, it exploits learned knowledge by choosing $a_t = \text{argmax}_a Q[s_t, a]$.
3. The environment executes action a_t, transitions to s_{t+1}, and returns reward r_t.
4. **Bellman Q-Learning Update:**
   $$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$
   where Learning Rate $\alpha = 0.10$ and Discount Factor $\gamma = 0.90$.
5. **Exploration Decay:** After each episode, ϵ decays via $\epsilon \leftarrow \max(0.01, \epsilon \times 0.9998)$. ϵ drops from 1.0 (pure exploration) to 0.01 (pure exploitation).
6. **Model Serialization:** The final converged Q-table is saved as JSON in `models/q_table.json` (81.8 KB), loading into memory with zero GPU/TensorFlow dependencies.

3. Empirical Evaluation & Benchmark Results

In `evaluate.py`, we evaluated the trained agent against a rigorous holdout test set of **5,000 unseen requests**. Here are the exact recorded metrics from `models/evaluation_results.json`:

Metric	Target Requirement	AegisRL Achieved	Evaluation Verdict
Attack Blocking Rate	≥ 95.0%	**99.37%** (1,733 / 1,744)	**EXCEEDED (+4.37%)**
Legitimate Access Approval	≥ 98.0%	**99.79%** (3,249 / 3,256)	**EXCEEDED (+1.79%)**
False Denial Rate	≤ 2.0%	**0.21%** (7 / 3,256)	**OPTIMAL (Minimal)**
Emergency Denial Rate	≤ 0.5%	**0.13%** (1 / 772)	**LIFE-SAFE**
Average Decision Latency	≤ 5.0 ms	**0.0166 ms** (16.6 µs)	**300x FASTER**
95th Percentile Latency (p95)	≤ 10.0 ms	**0.0198 ms** (19.8 µs)	**REAL-TIME GUARANTEE**
Cumulative Test Reward	Positive	**+71,363.0**	**CONVERGED OPTIMAL**

Detailed Action Breakdown Across Threat Scenarios:

Scenario Type	Samples	ALLOW	DENY	CHALLENGE	RESTRICT
NORMAL_ACCESS (Routine Care)	2,484	2,387 (96.1%)	6	35	56
EMERGENCY_ACCESS (Code-Blue)	772	770 (99.7%)	1	1	0
STOLEN_CREDENTIAL (External IP)	604	5	393 (65.1%)	202 (33.4%)	4
INSIDER_MISUSE (Browsing)	594	0	377 (63.5%)	20	197 (33.2%)
MASS_ACCESS (Bot Scraping)	546	2	493 (90.3%)	51	0

4. Backend Architecture & Supabase Cloud PostgreSQL Database

AegisRL integrates a high-performance Python FastAPI backend (`main.py`) with a 7-table normalized relational schema on Supabase PostgreSQL (`supabase_client.py`):

- **1. departments:** Stores hospital clinical units (Cardiology, Neurology, Oncology, Emergency, General Medicine) with head doctor and floor details.
- **2. users:** Contains hospital staff personas (Doctors, Nurses, Admins, Researchers) with user ID, department foreign key, role, and normalized clearance.
- **3. patients:** Maintains patient registry records (Patient ID, Code PAT-0001, full name, age, gender, assigned department).
- **4. medical_records:** Stores sensitive clinical files (Record ID, diagnosis, vitals, prescriptions, lab results, sensitivity tier LOW/MEDIUM/HIGH/CRITICAL_VIP).
- **5. access_requests:** Logs all incoming raw access attempts with timestamps, requester user ID, target patient ID, location, and frequency.
- **6. rl_decisions:** Stores the RL agent's decision outputs (Action ALLOW/DENY/CHALLENGE/RESTRICT, Q-Values vector, latency ms, state hash).
- **7. audit_logs:** HIPAA-compliant immutable audit trail recording complete context, outcome, and non-repudiation security hashes.

5. Complete Frontend Website Walkthrough (Every Page, Card & Button)

The AegisRL frontend is a responsive Single-Page Application (SPA). Here is a complete guide to all sections, buttons, and controls:

Top Navigation Bar & Header Controls

- **Sidebar Toggle Button:** On mobile/tablet screens, opens or collapses the left navigation drawer.
- **Breadcrumb Indicator:** Displays current active view path (e.g., Aegis-RL / Adaptive RL Decision Engine).
- **Theme Toggle Button (Sun / Moon):** Switches instantly between Light Mode (luminous porcelain with pastel accents) and Dark Mode (obsidian slate with glowing badges). Saved in localStorage.
- **Inference Latency Pill:** Displays real-time sub-millisecond Q-table lookup speed (< 0.02 ms).
- **Switch User Button:** Opens the Persona Switcher modal to switch active clinician persona (e.g., Dr. Vishnu Pandey in Cardiology vs. Dr. Priya Sharma in Emergency).

Page 1: Staff Dashboard (#dashboard)

- **Patient Records Button:** Navigates directly to the Patient Registry page.
- **New Request Button:** Navigates to the Request Access evaluation console.
- **Total Requests KPI Card:** Displays total historical requests logged in Supabase (10,000).
- **Approved KPI Card:** Displays total approved legitimate requests (7,249) with 99.79% legit rate.
- **Blocked KPI Card:** Displays total blocked malicious attempts (1,733) with 99.37% block rate.
- **Avg Latency KPI Card:** Displays sub-millisecond execution benchmark (0.016 ms).
- **Recent Decisions Table:** Displays live stream of the 8 latest evaluations with REQ ID, Patient, Scenario, Emergency status, Decision badge, and latency.
- **Full Audit Link (->):** Navigates to the Audit Logs page.

Page 2: Patient Registry (#patients)

- **Search Input Bar:** Filters patients in real-time by full name or patient code (e.g. PAT-0001).
- **Department Filter Chips:** Filter patients by clinic unit: All Depts, Cardiology, Neurology, Oncology, Emergency, General.
- **Patient Grid Cards:** Display patient avatar, PAT code, department badge, age/gender, and 'View Records ->' action.
- **Patient EHR Drawer:** Clicking any patient card slides out the right drawer showing all medical records for that patient stored in Supabase.
- **Request Access CTA inside Drawer:** Pre-fills the Request Access form with this specific patient's ID, department, and record ID.

Page 3: Adaptive Request Access Engine (#access)

- **Scenario Preset: Code-Blue Emergency:** Instantly configures parameters for a trauma emergency override (Doctor, cross-dept, high sensitivity, emergency active).
- **Scenario Preset: Routine Ward Care:** Pre-fills regular in-department patient rounding (Doctor, dept match, medium sensitivity, normal hours).
- **Scenario Preset: Off-Hours Audit:** Pre-fills suspicious off-site staff browsing (Staff, mismatch, VIP sensitivity, external IP).
- **Scenario Preset: Exfiltration Attack:** Pre-fills automated burst scraping (Staff, mismatch, high sensitivity, external IP, burst frequency).
- **Target Patient Card:** Shows active target patient initials, name, code, department, record ID, and 'Browse Patients ->' link.
- **Role Dropdown:** Select requester role (Doctor, Nurse, Admin/Staff, Researcher).
- **Sensitivity Dropdown:** Select record sensitivity tier (Low - Vitals, Medium - Labs, High - Psychiatric, Critical / VIP).
- **Shift Time Dropdown:** Select time window (Normal Working Hours 08-18 vs. Off-Hours / Night Shift).
- **Network Location Dropdown:** Select IP location (Internal Hospital LAN, Secure Clinical VPN, External / Unknown IP).
- **Query Frequency Dropdown:** Select request velocity (Normal 1-5/min, Elevated 6-20/min, Burst Threat >20/min).
- **Department Match Checkbox:** Toggles whether requester is assigned to the patient's specific clinical unit.
- **Emergency Override Checkbox:** Toggles active Code-Blue emergency protocol.

- **Evaluate Access Request Button:** Fires asynchronous API request to /predict, animates the 5-step pipeline, and renders the Decision Verdict Card.
- **5-Step Pipeline Animation:** Visual sequence showing step-by-step processing: Received -> Vectorized -> Profiled -> Q-Lookup -> Enforced.
- **Decision Verdict Card:** Renders large color-coded badge (ALLOW in green, DENY in red, CHALLENGE in amber, RESTRICT in cyan), action summary, explainability rationale callout, horizontal Q-Scores distribution bars, and database sync status.
- **View Unlocked Record Button:** Opens modal showing complete clinical diagnostics and prescriptions when ALLOW is granted.
- **View Masked Record Button:** Opens modal showing clinical vitals with patient PII redacted when RESTRICT is granted.

Page 4: Security Admin Console (#admin)

- **Launch Simulator Button:** Navigates directly to the Cyber Attack Simulator.
- **6 KPI Metric Cards:** Display Block Rate (99.37%), Legit Rate (99.79%), False Denial (0.21%), Emerg Denial (0.13%), Latency (0.016 ms), and Q-Reward (+71,363).
- **Action Distribution Chart (Chart.js):** Interactive doughnut chart showing breakdown between ALLOW, DENY, CHALLENGE, and RESTRICT.
- **Scenario Enforcement Chart (Chart.js):** Stacked bar chart visualizing policy actions across all 5 test scenarios.
- **Security Telemetry Stream:** Live table showing policy enforcement stream with filter chips (All, Allow, Deny, Challenge, Restrict).

Page 5: Cyber Threat Simulator (#simulator)

- **5 Scenario Tiles:** Clickable tiles (Normal Access, Emergency Access, Stolen Credentials, Insider Misuse, Mass Exfiltration) with 'Run ->' buttons to simulate live cyber attacks.
- **HTML5 Canvas Cyber Network:** Real-time animated visualization of network topology with live packets pulsing between External Threat, Gateway, EHR Vault, Wards, and Staff nodes.
- **Split Comparison Panel:** Compares what features the Agent Observes vs. Hidden Ground Truth (attack legitimacy, scenario type, safety outcome).
- **Rationale & Explainability Box:** Details the exact Q-learning reason why the action was taken.

Page 6: Access History & Audit Trail (#history)

- **Live Search Bar:** Searches audit logs by request ID, user ID, patient ID, or scenario.
- **Action Filter Pills:** Filter logs by ALL, ALLOW, DENY, CHALLENGE, RESTRICT.
- **Audit Trail Table:** Displays full audit history stored in Supabase with timestamp, user, patient, scenario, action, reward, and latency.
- **Inspect Button:** Opens the Audit Detail Modal showing the complete raw JSON security telemetry payload.

Persona Switcher Modal (#loginOverlay)

- **User Persona Pills:** List of staff members loaded live from Supabase. Clicking a persona updates AppState and switches active identity instantly.
- **Continue Button:** Closes the switcher modal and returns to the active page.

6. Project Viva Defense Guide & Presentation Script

If you or your teammates need to explain the project to a teacher, examiner, or viva panel, use this step-by-step explanation:

Step 1: The 30-Second Elevator Pitch

"Good morning sir/ma'am. Our project is AegisRL. It is an AI-powered zero-trust access control system for hospital electronic health records. Traditional access control uses fixed rules that either block doctors during emergency trauma care or fail to stop hackers using stolen passwords. We used Tabular Reinforcement Learning to build an agent that analyzes 7 real-time context features and decides in 0.016 milliseconds whether to ALLOW, DENY, CHALLENGE with MFA, or RESTRICT with data masking."

Step 2: Explaining How the Model was Trained

"We modeled the hospital access space as a Markov Decision Process with 1,152 discrete states. We formulated a custom reward function that heavily rewards saving lives during Code-Blue emergencies (+25.0) and blocking cyber threats (+15.0), while penalizing false alarms (-18.0) and emergency denials (-50.0). We trained the Q-learning agent over 35,000 episodes using the Bellman equation with an exploration decay strategy. The converged Q-table is saved as an 81 KB JSON lookup table."

Step 3: Explaining the Results & Accuracy

"We tested the model on 5,000 unseen requests. It achieved a 99.37% attack blocking rate, 99.79% legitimate approval rate, and only 0.13% emergency denial rate. Because we used Tabular Q-learning rather than heavy neural networks, our inference takes only 16 microseconds, which is 300 times faster than deep learning models and 100% explainable."

Step 4: Explaining the Technology Stack

"For the backend, we used Python with FastAPI and Uvicorn. For the database, we connected Supabase PostgreSQL with 7 relational tables for HIPAA-compliant audit logging. For the frontend, we built an interactive web dashboard with dual Light/Dark modes, an HTML5 canvas network attack simulator, and Chart.js telemetry charts."